

Platform as a Product

A Deep Dive into the Technical Foundations

Abby Bangser (she/her)
abby@syntasso.io
Syntasso.io / Kratix.io

My first project moving to the UK

What started as a simple stop
gap over the holidays, turned
into a day 2 planning exercise

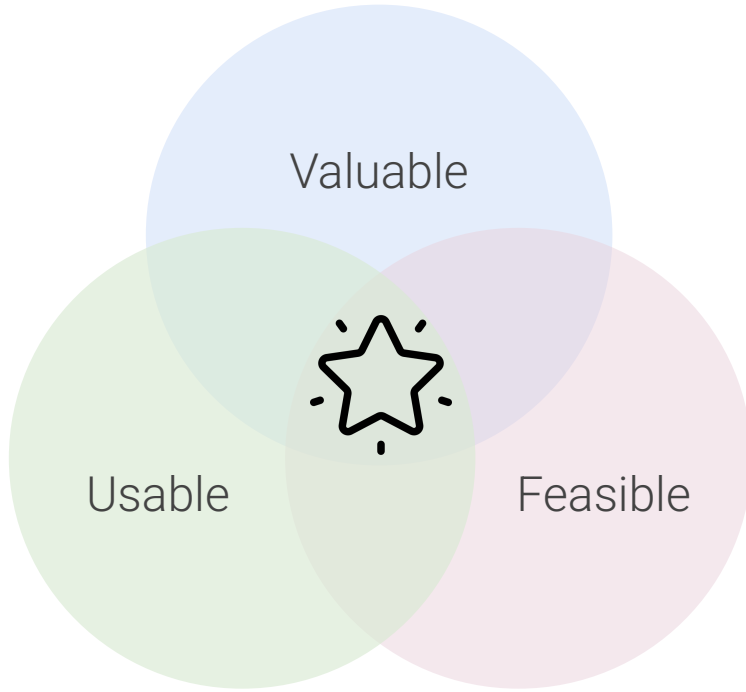


Before I got there, the team had...

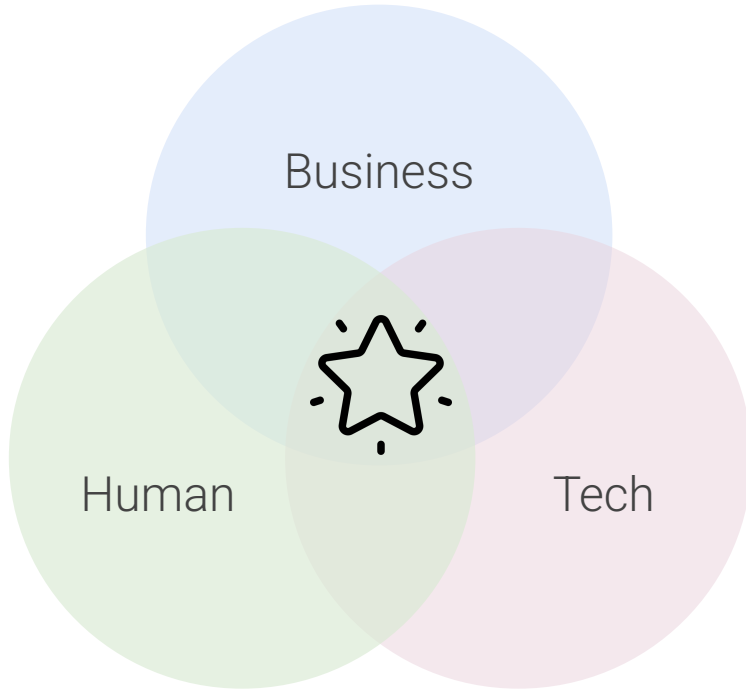
- User research
- Product design workshops
- Architecture board approvals
- MVP feature identification
- Pair programming
- Infrastructure as Code
- Extensive automated testing
- Security and compliance reviews
- CI/CD to testing environments
- ...

What still needed thinking...

- Trunk based delivery when in production
 - Change Approval Board (CAB) processes with CD
 - Secure environment production debugging
 - First and second-line support
-

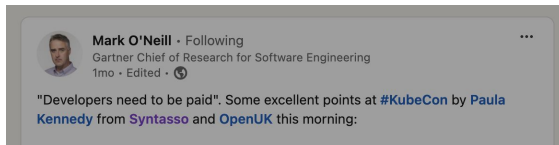


Products are more than a project



Products require a
balanced
multi-disciplinary
approach

Reported failure cases are almost all when technology overpowers



It's natural for a technology to go into the Trough, and indeed we are seeing the "Build it and they will come" issues that Paula mentions too.

- She said that Platform Engineering was at the top of the Peak of Inflated Expectations in 2023, and is headed for the Trough of Disillusionment this year. Indeed we had Platform Engineering close to the top of our Hype Cycle for Software Engineering last year, expertly curated by Dave Mickle, but it was never going to be as high on the Peak as Generative AI or AI Coding Assistants. It's natural for a technology to go into the Trough, and indeed we are seeing the "Build it and they will come" issues that Paula mentions too.

- "Platforms going stale" is a thing. Expect to see more of this.

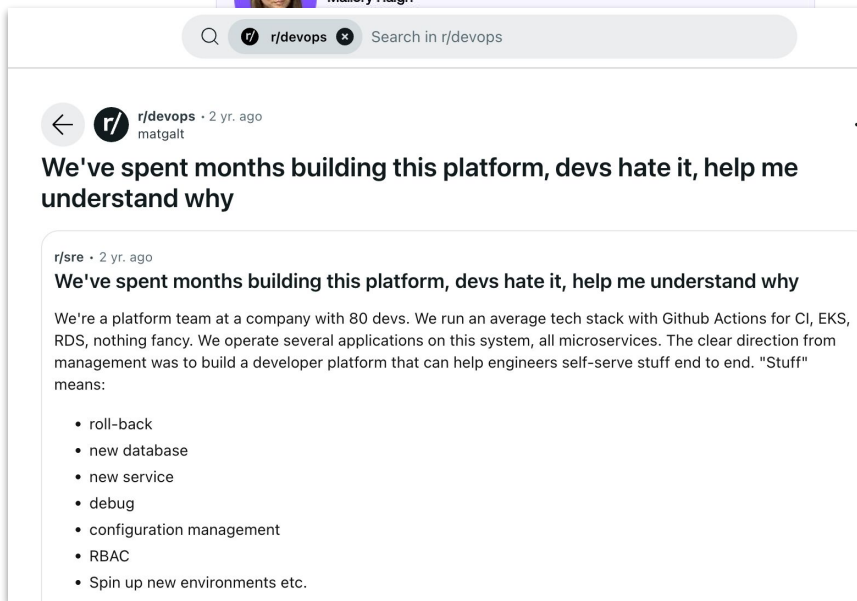
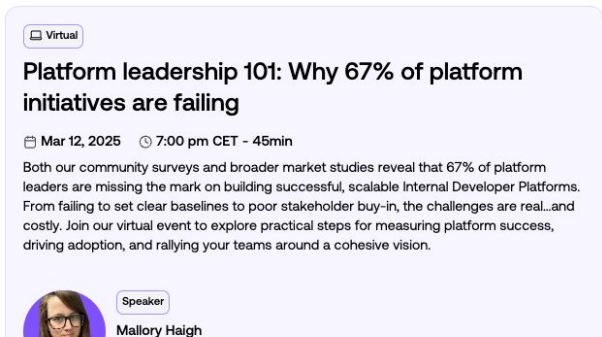
- Developers should be paid, and, post ZIRP, tech companies are focused on revenue. So we are seeing license changes, with Redis just the latest today.

- "I would look closely at an open source project which is mainly government funded". Governments funding open source projects seems like a great idea, but this is a great point.

Overall, very good points and it was good to see someone grasp the nettle of the recent license changes in the world of open source.

👍👍 You and 36 others

3 comments • 2 reposts



In platforms, products need to balance more business and user

<https://www.youtube.com/watch?v=xTUePYLtsyE> **goto**

Adopt a Product Mindset...

- Negotiate **Outcomes** based on **Business Requirements** (Impact)
- Build **Features** based on **Functional Requirements**
- Agree on **Platform Roadmap** capabilities vs priorities
- Establish and nurture a **continuous feedback** culture



Platforms must advocate for their platform product and "market" it to internal teams

Platform as a Product

Manuel Pais
Co-Author Team Topologies

PLATFORM CON 22

<https://www.youtube.com/watch?v=b8YHCDMxqfg>



DEVONX United Kingdom

- You are frustrated due to the platform empathy gap
- Large empathy gap means low adoption
- Bridge the gap and make adoption your north star metric
- Fix the reasons why developers **don't** adopt your platform

SYNTASSO

<https://www.youtube.com/watch?v=t3p1fLAZEg4>



Boosting Developer Platform Teams with Product Thinking - Samantha Coffman, Spotify

https://www.youtube.com/watch?v=Z_KCOcolLI

Deploy Empathy: Everyone should build customer empathy by talking to them, often

... I realise that I am not needed in the meeting that I was invited to, and the incident didn't have anything to do with my team

Platform Engineering EUROPE



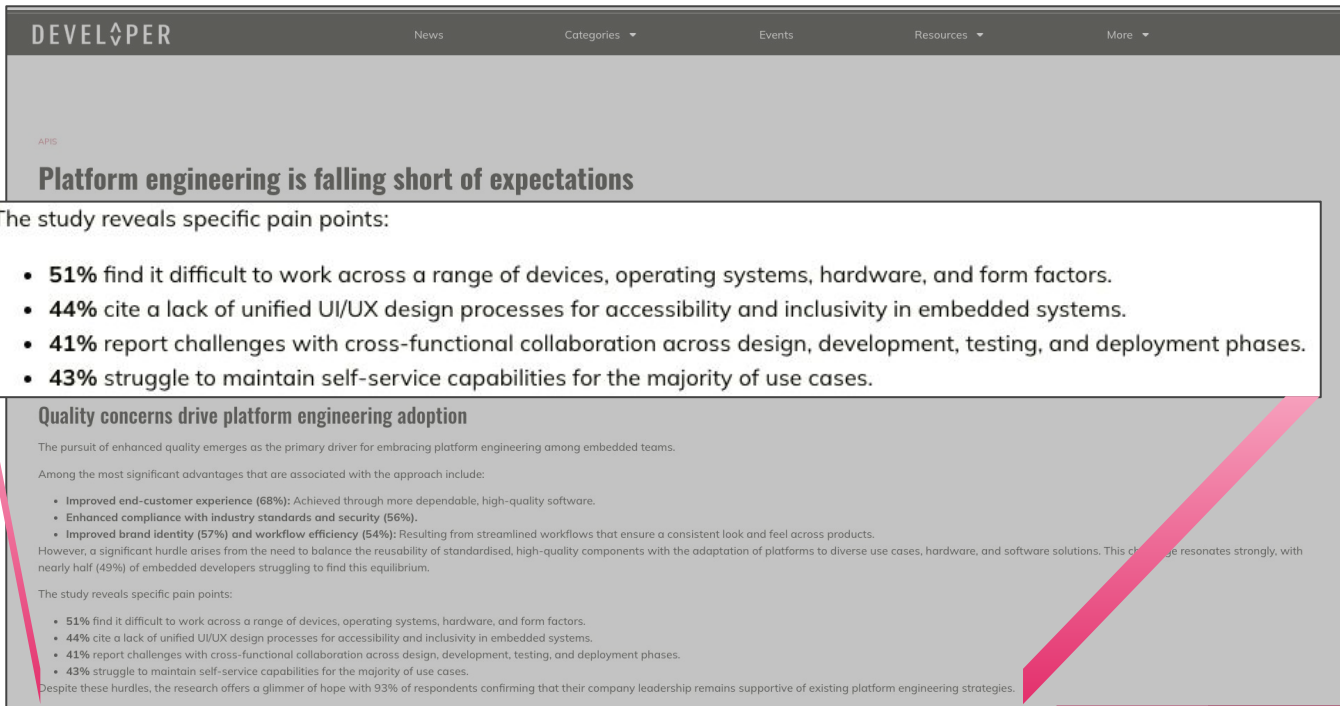
WHITE PAPER

Pivotal Platform Solutions: Why You Should Treat Platform as a Product

By Joe Fitzgerald and Colin Humphreys

<https://wearemomentum.ai/whitepapers/2017-platform-as-a-product-whitepaper.pdf>

That shouldn't make us complacent with technology



DEVLOPER News Categories Events Resources More

APIS

Platform engineering is falling short of expectations

The study reveals specific pain points:

- **51%** find it difficult to work across a range of devices, operating systems, hardware, and form factors.
- **44%** cite a lack of unified UI/UX design processes for accessibility and inclusivity in embedded systems.
- **41%** report challenges with cross-functional collaboration across design, development, testing, and deployment phases.
- **43%** struggle to maintain self-service capabilities for the majority of use cases.

Quality concerns drive platform engineering adoption

The pursuit of enhanced quality emerges as the primary driver for embracing platform engineering among embedded teams.

Among the most significant advantages that are associated with the approach include:

- **Improved end-customer experience (68%)**: Achieved through more dependable, high-quality software.
- **Enhanced compliance with industry standards and security (56%)**.
- **Improved brand identity (57%) and workflow efficiency (54%)**: Resulting from streamlined workflows that ensure a consistent look and feel across products.

However, a significant hurdle arises from the need to balance the reusability of standardised, high-quality components with the adaptation of platforms to diverse use cases, hardware, and software solutions. This challenge resonates strongly, with nearly half (49%) of embedded developers struggling to find this equilibrium.

The study reveals specific pain points:

- **51%** find it difficult to work across a range of devices, operating systems, hardware, and form factors.
- **44%** cite a lack of unified UI/UX design processes for accessibility and inclusivity in embedded systems.
- **41%** report challenges with cross-functional collaboration across design, development, testing, and deployment phases.
- **43%** struggle to maintain self-service capabilities for the majority of use cases.

Despite these hurdles, the research offers a glimmer of hope with 93% of respondents confirming that their company leadership remains supportive of existing platform engineering strategies.

Platforms technology affects business and users

- Features like failure recovery are user facing
- Scaling isn't just a non-functional requirement
- Updating is a core user feature





Lack of technical investment in
supporting these features
leads to decay over time

Decay is the impact of unchecked platform tech debt

Field of (broken) dreams: If you build it, they may not come

Lack of adoption (or declining adoption) is a common sign of platform decay

Steps you can take to stop it

- Apply product mindset including
 - Survey developers to identify biggest blockers and pain points (and fix them)
 - Iterate with small releases to meet needs
- Assign a dedicated “enabling” team to onboard teams onto the platform
- Market the value provided to existing

Platform money pit: The dangers of high cost and low impact

Platform decay can manifest as low ROI, often caused by high platform maintenance costs

Why?

- Project rather than a product implementations
- Scoping too low level, trying to create an “internal AWS/GCP/Azure”
- Big bang releases resulting in a big bang with expensive cleanup
- No clear ownership, budget, roadmap alignment with company and user needs

Steps you can take to stop it

- Apply product mindset including
 - Identify, scope, and deliver a platform-based fix for one problem/team

Avoiding multiple migrations: Delivering value incrementally

Managing multiple platform migrations is often a trailing indicator of platform decay

Why?

- Project mindset funds only big bang solutions / migrations
- Lack of focus on defining a platform API and providing X-as-a-service

Steps you can take to stop it

- Apply a product mindset including
 - Continuous evolution over single release
 - Plan value driven migrations rather than be forced to complete EOL driven migrations

KRATIX.IO



Tech debt is even harder to prioritise for internal...

- No clear revenue gain / ROI
- Unclear path to adoption
- Enticing off-the-shelf solutions

Tech debt is even harder to prioritise for internal...

- No clear revenue gain / ROI
- Unclear path to adoption
- Enticing off-the-shelf solutions

...yet high leverage work wins markets

- Compete on differentiation
 - Costs reduces overall revenue
 - Internal tooling impacts reputation
-

Cutting to the chase

- **Internal tools aren't special.** Some context may apply, but software is software.
- **It is never the right time.** Invest early in the structures so each change is easier.
- **Platforms are force multipliers.** Design for expansion not single track growth.



Defining the platform context

→ Provide a uniform delivery mechanism



Platform



Defining the platform context

- Provide a uniform delivery mechanism
- For anything but uniform services
- Platform services have the same cognitive load challenges as business applications





The field of platform engineering is less forgiving.

Platforms are amongst the most valuable ideas...and at the same time the most difficult things to do well.

Dave Farley & Gregor Hohpe
<https://youtu.be/fLZRk5vzMWY>

The successful platforms
are "just" software.

This means sharing
learning with decades of
software delivery.





Introducing the walking skeleton



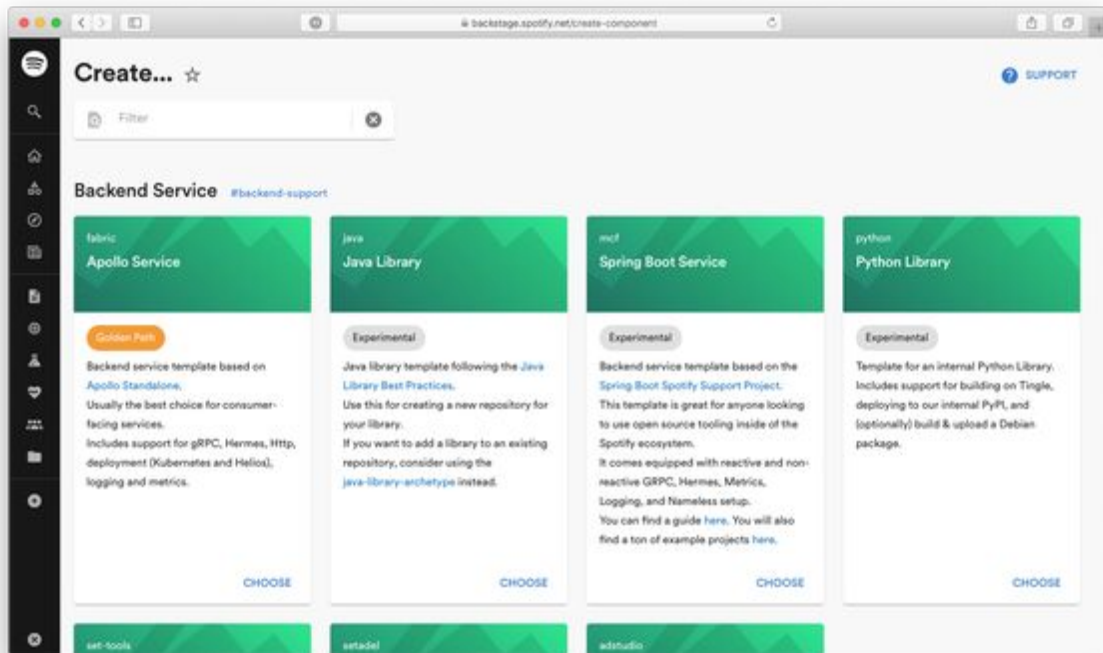
Applying ~~Introducing~~ the walking skeleton

What is a walking skeleton

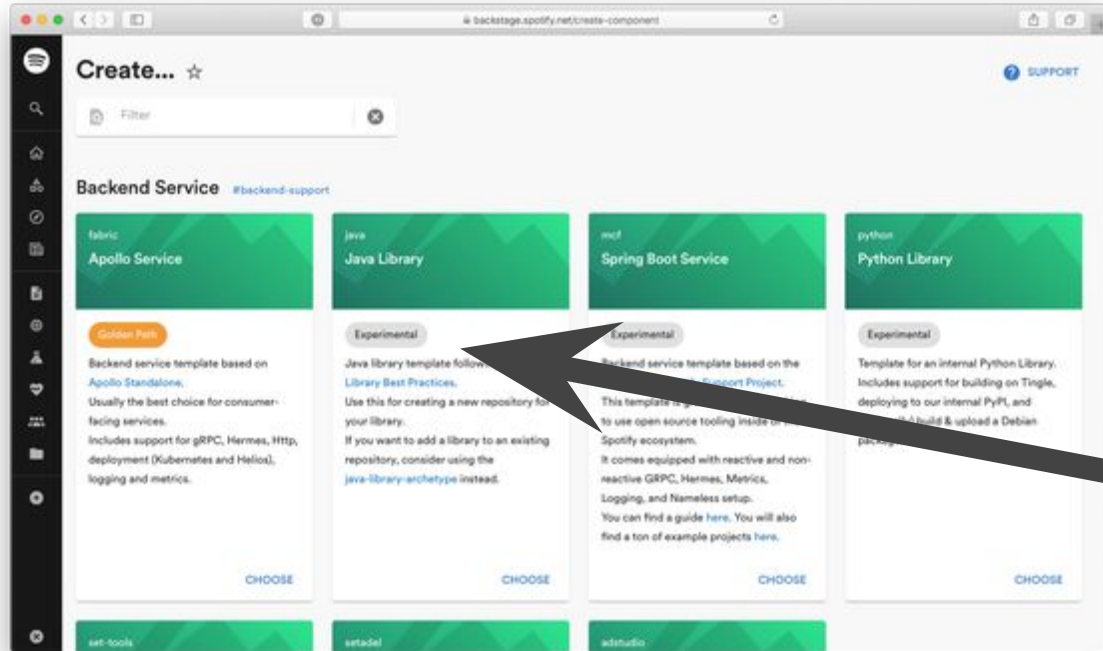
"a **minimal, but complete and deployable**, end-to-end version of a software system that demonstrates the fundamental architecture and core functionality, allowing for continuous integration, automated testing, and **early feedback to validate technical assumptions** and steer development"



Define the vision



Identify the parts



Build the frame

- Hello world capability
- Capabilities as APIs
- Accessible interface



Start walking

- Initial test suites
- Operability framework
- End-to-end update



Finish the look

- Contributor model
- Enable optionality
- Regular validation



Build the frame

- Hello world capability
- Capabilities as APIs
- Accessible interface

Start walking

- Initial test suites
- Operability framework
- End-to-end update

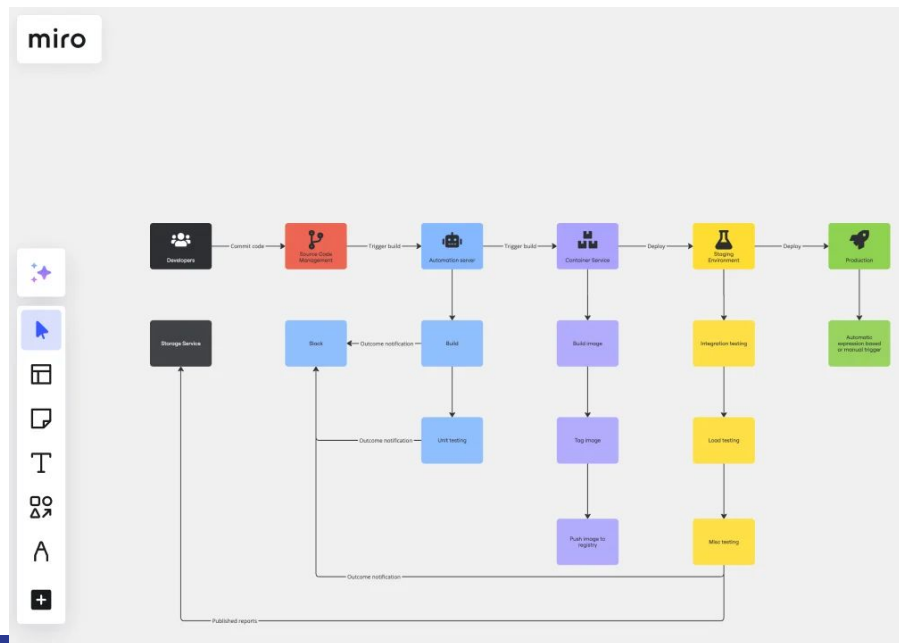
Finish the look

- Contributor model
- Enable optionality
- Regular validation



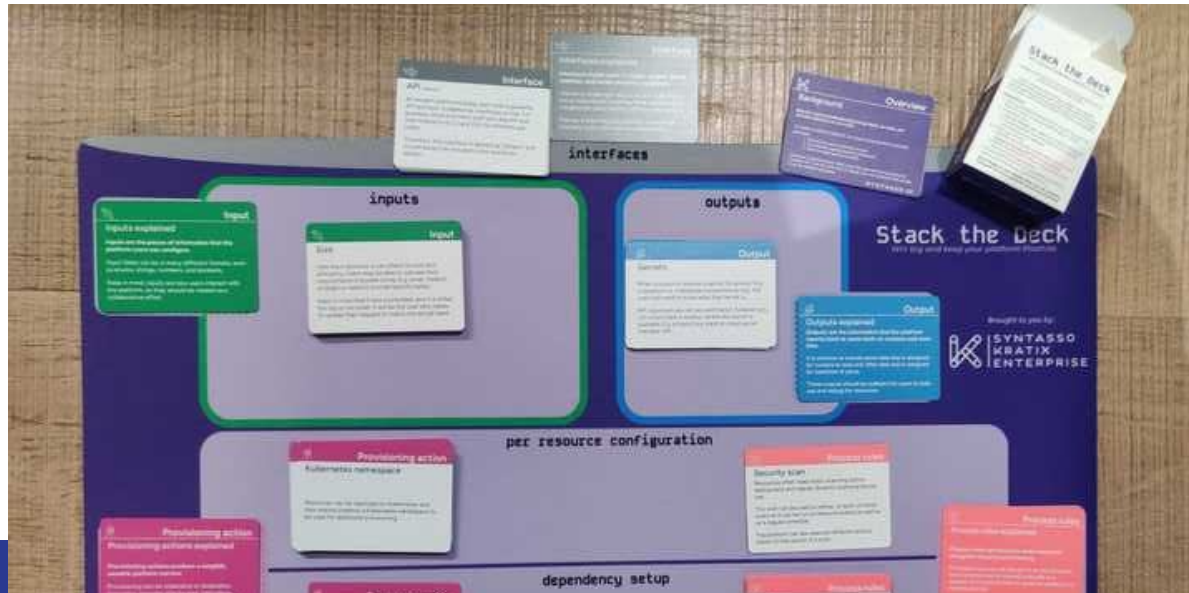
Build the frame: Hello world capability

→ Automate a simple capability including business processes



Build the frame: Hello world capability

- Automate a simple capability including business processes
- Define user interface inputs and outputs



Build the frame: **Hello world capability**

- Automate a simple capability including business processes
- Define user interface inputs and outputs
- Enable user CRUD actions



Build the frame: Capabilities as APIs

→ Decouple user experience from implementation language

```
1 module "bucket-store" {  
2   source = "terraform-aws-modules/s3-bucket/aws"  
3   version = "3.15.1"  
4  
5   bucket = "${local.env_name}-bucket-store"  
6   acl    = "private"  
7  
8   force_destroy      = true  
9   control_object_ownership = true  
10  object_ownership    = "ObjectWriter"  
11  
12  versioning = {  
13    enabled = false  
14  }  
15 }
```



```
1 apiVersion: marketplace.kratix.io/v1alpha1  
2 kind: bucket  
3 metadata:  
4   name: cdn  
5   namespace: team-backend  
6 spec:  
7   bucketConfig:  
8     public: false  
9     versioning: true  
10    encryption: SSE-KMS  
11    region: us-west-2  
12    objectLocking: false  
13
```

Build the frame: Capabilities as APIs

- Decouple user experience from implementation language
- Faster flow of value by removing user impact for all changes



Figure 7.5: Primary Interaction Modes for the Four Fundamental Team Topologies

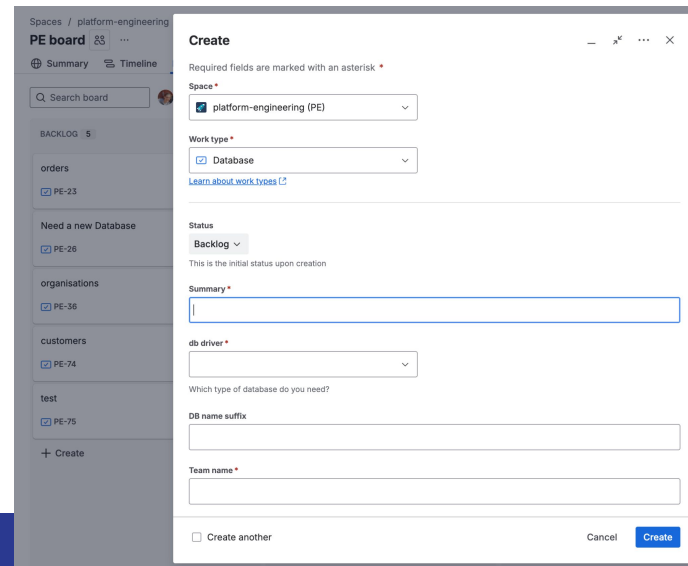
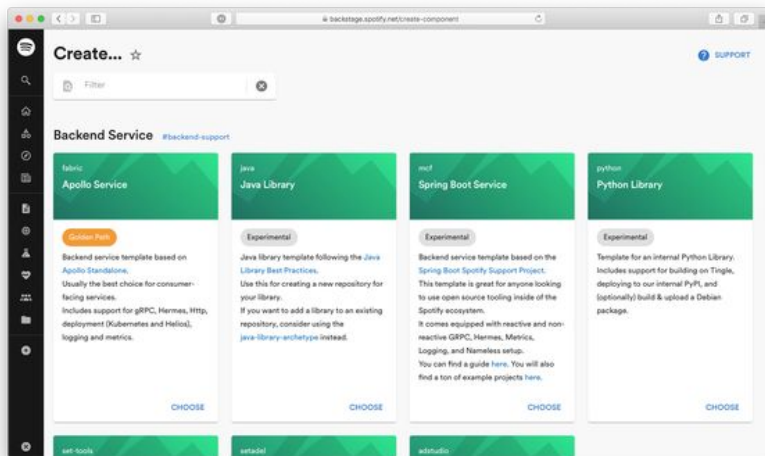
Build the frame: Capabilities as APIs

- Decouple user experience from implementation language
- Faster flow of value by removing user impact for all changes
- Build with intentional services and org design
 - ◆ DDD and Team Topologies to determine capability shape, size, and interaction
 - ◆ Include engineers early with API design and evolution experience



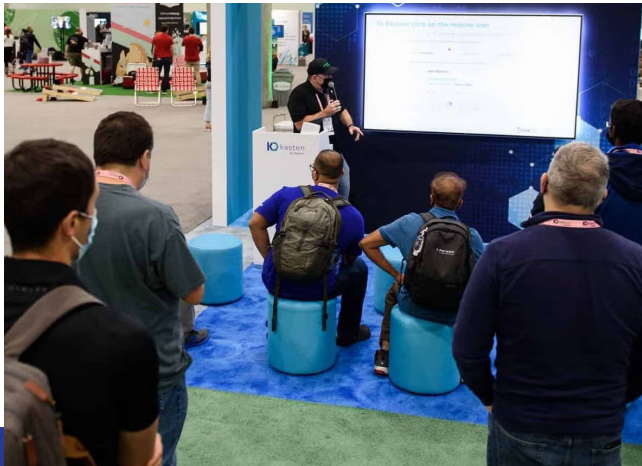
Build the frame: Accessible interface

- Unified platforms require an implementation agnostic marketplace
- This is not the *only* interface, but needs to be a common one



Build the frame: Accessible interface

- Unified platforms require an implementation agnostic marketplace
- This is not the *only* interface, but needs to be a common one
- Building a brand is easier with visuals



Build the frame

- Hello world capability
- Capabilities as APIs
- Accessible interface



Start walking

- Initial test suites
- Operability framework
- End-to-end update



Finish the look

- Contributor model
- Enable optionality
- Regular validation



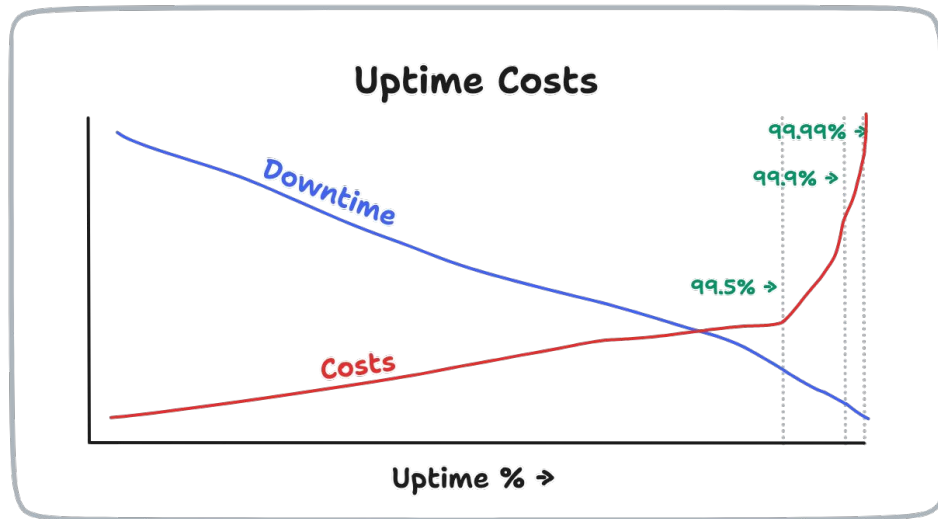
Start walking: **Initial test suites**

- Brainstorm risks from cyber attack through to incorrect logic
- Batch into suites keeping fastest/cheapest possible feedback loop
- Introduce each suite, don't try to cover each risk



Start walking: Operability framework

- Operability is support but also product feedback
- Define SLIs and set SLOs to pragmatic levels



ITZY SABO

<https://world.hey.com/itzy/how-much-uptime-can-i-afford-3130e605>

Start walking: Operability framework

- Operability is support but also product feedback
- Define SLIs and set SLOs to pragmatic levels
- Exploratory testing/gamedays to validate



Start walking: End-to-end update

→ Adoption depends on distribution. Retention depends on updates.

```
curl -LO https://dl.k8s.io/release/v1.34.0/bin/linux/amd64/kubectl
```

Linux Debian/Ubuntu Red Hat/Fedora/CentOS macOS Windows

1. Confirm that you have a supported version of Python:

- To check your current Python version, run `python3 -V` or `python -V`. Supported versions are Python 3.9 to 3.13.

```
$ brew --help
Example usage:
brew search TEXT|/REGEX/
brew info [FORMULA|CASK...]
brew install FORMULA|CASK...
brew update
brew upgrade [FORMULA|CASK...]
brew uninstall FORMULA|CASK...
brew list [FORMULA|CASK...]
```

Start walking: **End-to-end update**

- Adoption depends on distribution. Retention depends on updates.
- Upgrades are best owned by those who see value in their upkeep
 - ◆ Be proactive with dependency management
 - ◆ Reduce blast radius for each upgrade
 - ◆ Make validating contracts on upgrade low cost
- Internal customers are "2nd party" and need support



Build the frame

- Hello world capability
- Capabilities as APIs
- Accessible interface



Start walking

- Initial test suites
- Operability framework
- End-to-end update



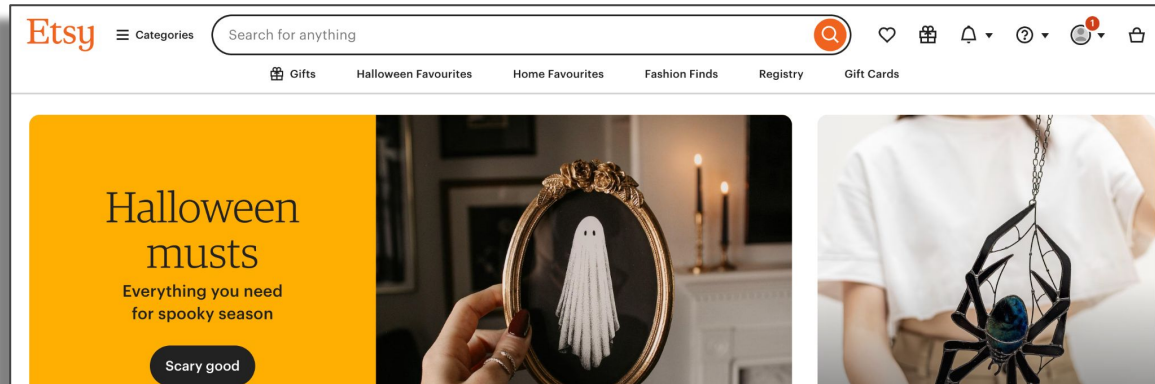
Finish the look

- Contributor model
- Enable optionality
- Regular validation



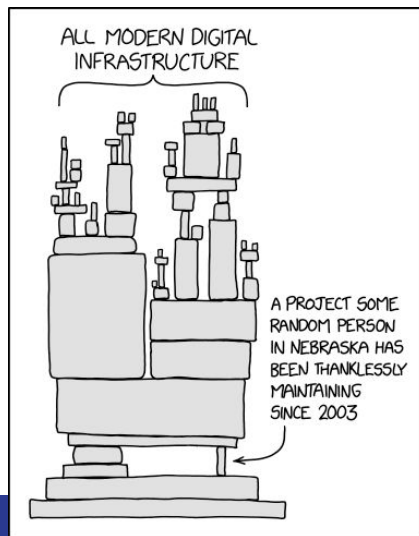
Dress up: Contributor model

- Platforms are not the new ops, a central team can't support all requirements
- Contributions are a platform feature, reduce their undifferentiated building



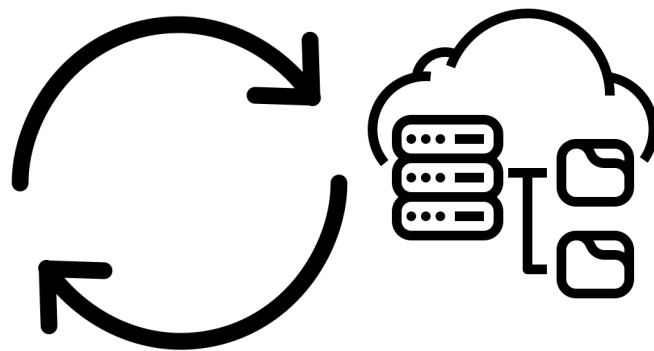
Dress up: Contributor model

- Platforms are not the new ops, a central team can't support all requirements
- Contributions are a platform feature, reduce their undifferentiated building
- Innersourcing should be empowering experts, not relying on unpaid labour



Start walking: Regular validation

→ Drift isn't just initial expectations, expectations can also evolve



Start walking: Regular validation

- Drift isn't just initial expectations, expectations can also evolve
- Platforms should expect, identify, and mitigate divergence



Start walking: Regular validation

- Drift isn't just initial expectations, expectations can also evolve
- Platforms should expect, identify, and mitigate divergence
- All drift is drift, apply common patterns to build resilience in



Dress up: **Enable optionality**

- Validate resource behaviour not implementation
- Provide self-checks for clear expectations to reduce shadow IT
- This is a cheat to building a platform people want



Don't forget that tech is *part of* product

- Build product roadmaps
not project deadlines
- Research use cases by
engaging with users
- Intentionally
market/evangelise



Wrapping up

- Products are multidisciplinary.
- Internal tools aren't special.
- Set groundwork early.
- Platforms are more than features.



Wrapping up

- **Products are multidisciplinary.** It is tech's job to enable human & business side.
- **Internal tools aren't special.** Some context may apply, but software is software.
- **Set groundwork early.** It is easier to add to something than to define it. Get the skeleton built with the right support systems as early as possible.
- **Platforms are more than features.** Centralise the ability to self-serve but democratise capability building.



Thank you

Abby Bangser (she/her)
abby@syntasso.io
Syntasso.io / Kratix.io

goto;

QUESTIONS?

ASK ME IN THE APP

goto;

RATE THIS SESSION

YES, YOU'LL GET A REWARD